

Documentação de Projeto para o Sistema de Moeda Estudantil

Versão 1.0

Projeto de sistema elaborado pelo(s) aluno(s) **Matheus Santos Ribeiro** como parte da disciplina Projeto de Software.

Data de Criação: 13 de Novembro de 2025

Tabela de Conteúdo

- 1 Introdução
 - 2 Modelos de Usuário e Requisitos
 - 2.1 Descrição de Atores
 - 2.2 Modelo de Casos de Uso e Histórias de Usuários
 - 2.3 Diagrama de Sequência do Sistema e Contrato de Operações
 - 3 Modelos de Projeto
 - 3.1 Arquitetura
 - 3.2 Diagrama de Componentes e Implantação
 - 3.3 Diagrama de Classes
 - 3.4 Diagramas de Sequência
 - 3.5 Diagramas de Comunicação
 - 3.6 Diagramas de Estados
 - 4 Modelos de Dados
-

1. Introdução

Este documento apresenta a elaboração e revisão dos modelos de domínio e modelos de projeto para o **Sistema de Moeda Estudantil**. O sistema tem como objetivo reconhecer e valorizar o mérito estudantil por meio de uma moeda virtual, distribuída por professores e utilizada por alunos para resgatar produtos e vantagens oferecidas por empresas parceiras [1].

O projeto segue a arquitetura **MVC (Model–View–Controller)** e utiliza tecnologias modernas de desenvolvimento web, como **TypeScript**, **JavaScript (ES6+)**, **HTML5** e **CSS3**.

[1]. A persistência de dados é gerenciada pelo **Prisma ORM** com um banco de dados **PostgreSQL** [2].

2. Modelos de Usuário e Requisitos

2.1 Descrição de Atores

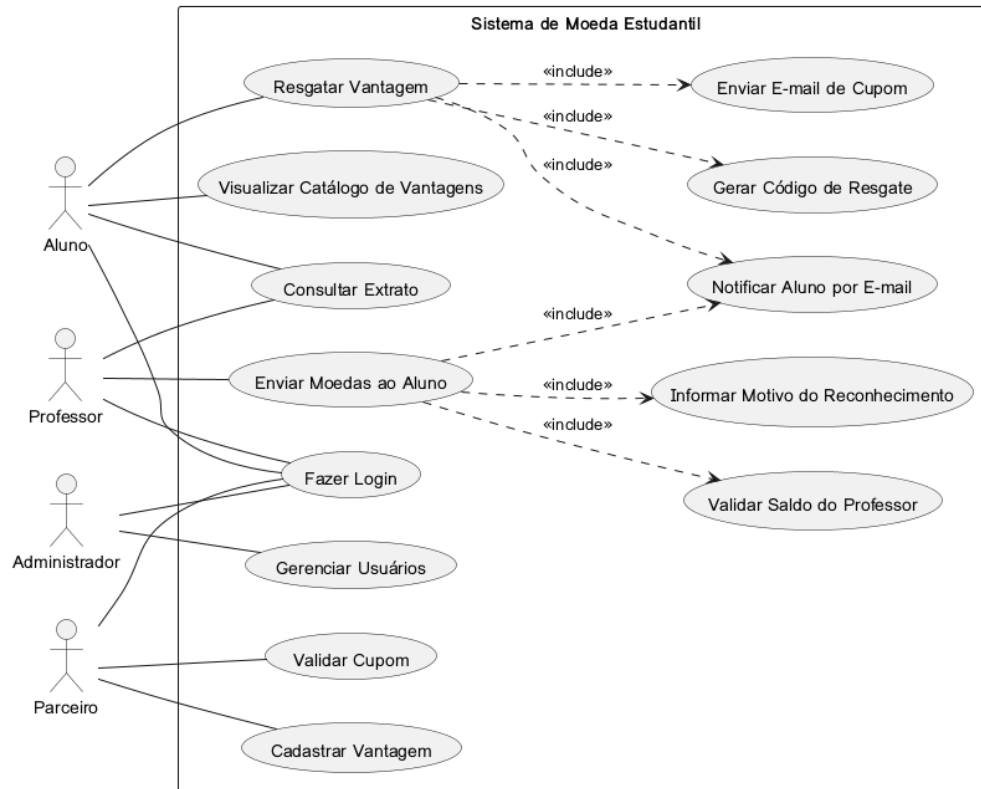
O sistema define quatro papéis principais de usuário, conforme o esquema de banco de dados e a descrição do projeto [1] [2]:

Ator	Papel no Sistema	Descrição
Aluno	<u>ALUNO</u>	Usuário principal que recebe moedas de professores e as utiliza para resgatar vantagens oferecidas por empresas parceiras. Pode consultar extrato e saldo.
Professor	<u>PROFESSOR</u>	Usuário responsável por enviar moedas aos alunos como forma de reconhecimento, registrando o motivo. Pode consultar extrato e saldo total.
Empresa Parceira	<u>PARCEIRO</u>	Entidade que cadastra vantagens (produtos ou descontos) e valida os cupons de resgate gerados pelo sistema.
Administrador	<u>ADMIN</u>	Usuário com privilégios de gerenciamento do sistema (não detalhado no escopo do projeto, mas presente no modelo de dados).

2.2 Modelo de Casos de Uso e Histórias de Usuários

O diagrama de Casos de Uso (Figura 1) ilustra as principais interações dos atores com o sistema.

Figura 1: Diagrama de Casos de Uso



Os principais Casos de Uso são:

ID	Caso de Uso	Atores	Descrição
UC-01	Fazer Login	Aluno, Professor, Parceiro	Autenticar credenciais para acesso ao sistema.
UC-02	Enviar Moedas ao Aluno	Professor	Transferir moedas para um aluno, informando o motivo do reconhecimento.
UC-03	Resgatar Vantagem	Aluno	Trocar moedas por uma vantagem cadastrada, gerando um cupom de resgate.
UC-04	Validar Cupom	Parceiro	Inserir o código do cupom para confirmar o resgate e atualizar o status.
UC-05	Consultar Extrato	Aluno, Professor	Visualizar o histórico detalhado de transações (envio, recebimento, resgate).
UC-06	Visualizar Catálogo de Vantagens	Aluno	Navegar e selecionar as vantagens disponíveis para resgate.
UC-07	Cadastrar Vantagem	Parceiro	Inserir novas ofertas de produtos ou serviços no catálogo.

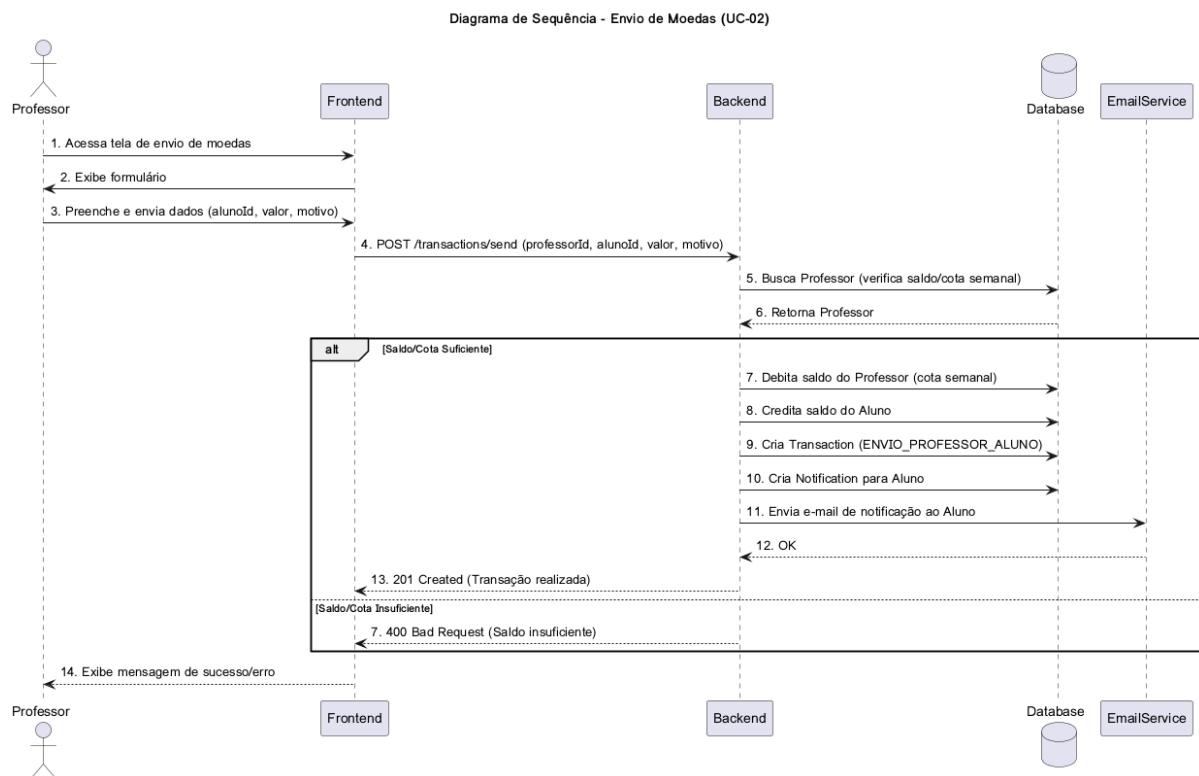
2.3 Diagrama de Sequência do Sistema e Contrato de Operações

Abaixo, são apresentados três Diagramas de Sequência do Sistema (DSS) para os Casos de Uso mais críticos, seguidos dos contratos de operação correspondentes.

DSS 1: Envio de Moedas (UC-02)

O diagrama (Figura 2) detalha o fluxo de interação entre o Professor, o Frontend, o Backend, o Banco de Dados e o Serviço de E-mail para a transferência de moedas.

Figura 2: Diagrama de Sequência - Envio de Moeda



Contrato de Operação: enviarMoedas(professorId, alunoId, valor, motivo)

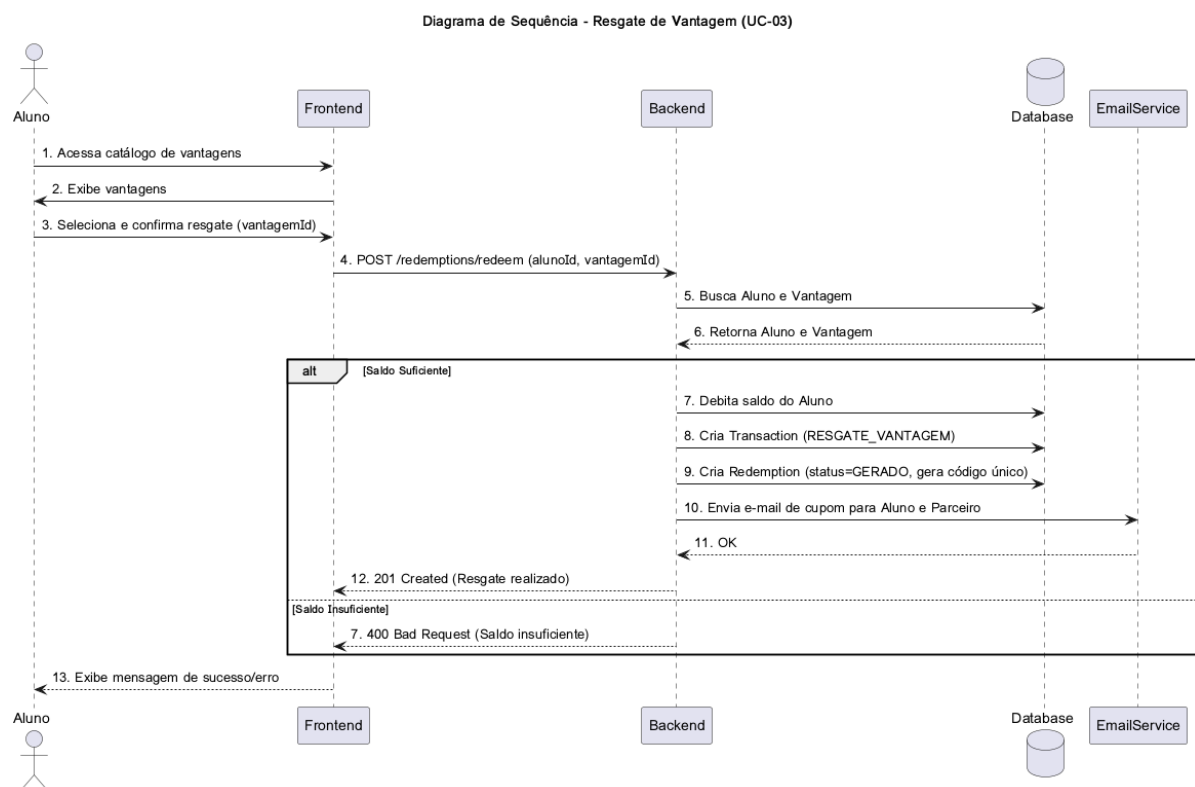
Campo	Descrição
Operação	<u>POST /transactions/send</u>
Referências cruzadas	UC-02: Enviar Moedas ao Aluno
Pré-condições	O Professor deve estar autenticado. O Professor deve ter saldo suficiente (cota semanal). O Aluno deve existir.

Campo	Descrição
Pós-condições	Uma transação do tipo <u>ENVIO PROFESSOR ALUNO</u> é criada. O saldo do Professor é debitado (cota semanal). O saldo do Aluno é creditado. Uma notificação é criada para o Aluno. Um e-mail de notificação é enviado ao Aluno.

DSS 2: Resgate de Vantagem (UC-03)

O diagrama (Figura 3) mostra o fluxo para o Aluno resgatar uma vantagem, envolvendo a verificação de saldo e a geração de cupom.

Figura 3: Diagrama de Sequência - Resgate de Vantagem



Contrato de Operação: resgatarVantagem(alunoId, vantagemId)

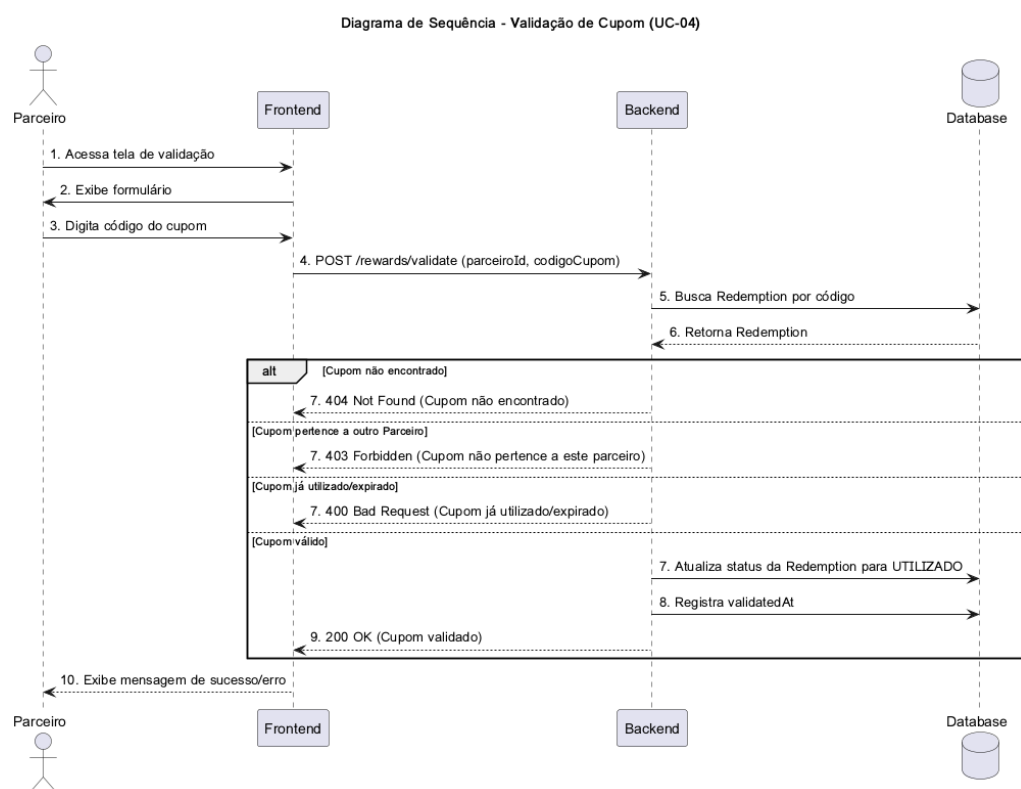
Campo	Descrição
Operação	<u>POST /redemptions/redeem</u>
Referências cruzadas	UC-03: Resgatar Vantagem
Pré-condições	O Aluno deve estar autenticado. A Vantagem deve existir e estar disponível. O Aluno deve ter saldo suficiente para o custo da Vantagem.

Campo	Descrição
Pós-condições	Uma transação do tipo <u>RESGATE VANTAGEM</u> é criada. O saldo do Aluno é debitado. Uma nova <u>Redemption</u> (resgate) é criada com status <u>GERADO</u> e um código único. Um e-mail com o cupom é enviado ao Aluno e ao Parceiro.

DSS 3: Validação de Cupom (UC-04)

O diagrama (Figura 4) ilustra o processo de validação do cupom pelo Parceiro, garantindo que o cupom seja único e não tenha sido utilizado ou expirado.

Figura 4: Diagrama de Sequência - Validação de Cupom pelo Parceiro



Contrato de Operação: validarCupom(parceiroId, codigoCupom)

Campo	Descrição
Operação	<u>POST /rewards/validate</u>
Referências cruzadas	UC-04: Validar Cupom

Campo	Descrição
Pré-condições	O Parceiro deve estar autenticado. O código do cupom deve ser válido (existir). O cupom deve pertencer ao Parceiro. O cupom não deve ter sido utilizado (<u>UTILIZADO</u>) ou expirado (<u>EXPIRADO</u>).
Pós-condições	O status da <u>Redemption</u> (resgate) é alterado para <u>UTILIZADO</u> . A data de validação (<u>validatedAt</u>) é registrada. O Parceiro recebe a confirmação de validação.

3. Modelos de Projeto

3.1 Arquitetura

O sistema adota a arquitetura **MVC (Model-View-Controller)**, o que facilita a separação de responsabilidades e a manutenção do código.

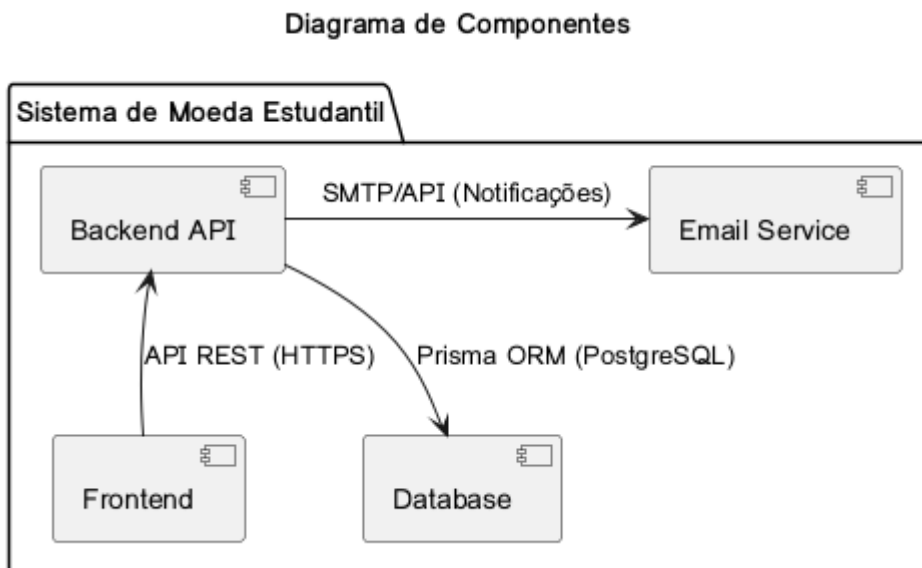
- **Model (backend-sistema-moeda/prisma):** Responsável pela lógica de negócios e persistência de dados, utilizando o Prisma ORM para interagir com o banco de dados PostgreSQL.
- **View (src/pages, src/components):** A interface do usuário, construída com React/TypeScript, que exibe os dados e captura as interações do usuário.
- **Controller (backend-sistema-moeda/src):** Intermedeia as requisições do usuário (View) e as regras de negócio (Model), coordenando o fluxo de dados e a lógica de aplicação.

Além disso, a arquitetura é distribuída, com um **Frontend** (React/Vite) e um **Backend** (Node.js/TypeScript) separados, comunicando-se via API REST.

3.2 Diagrama de Componentes e Implantação

O Diagrama de Componentes (Figura 5) mostra a estrutura modular do sistema, e o Diagrama de Implantação (implícito no diagrama de componentes) sugere a distribuição em diferentes ambientes de execução.

Figura 5: Diagrama de Componentes



Os principais componentes são:

- **Frontend (Interface do Usuário):** Componente web (React/Vite) que lida com a apresentação e interação do usuário.
- **Backend (Serviço de Aplicação):** Componente principal que contém a lógica de negócios e a API REST.
- **Database (PostgreSQL):** Componente de persistência de dados, gerenciado pelo Prisma ORM.
- **Email Service (Serviço de E-mail):** Componente externo responsável pelo envio de notificações transacionais (e-mails de envio de moedas e cupons).

3.3 Diagrama de Classes

O Diagrama de Classes (Figura 6) representa a estrutura estática do sistema, mostrando as classes, seus atributos, métodos e os relacionamentos entre elas. O diagrama é derivado do [schema.prisma](#) e das regras de negócio.

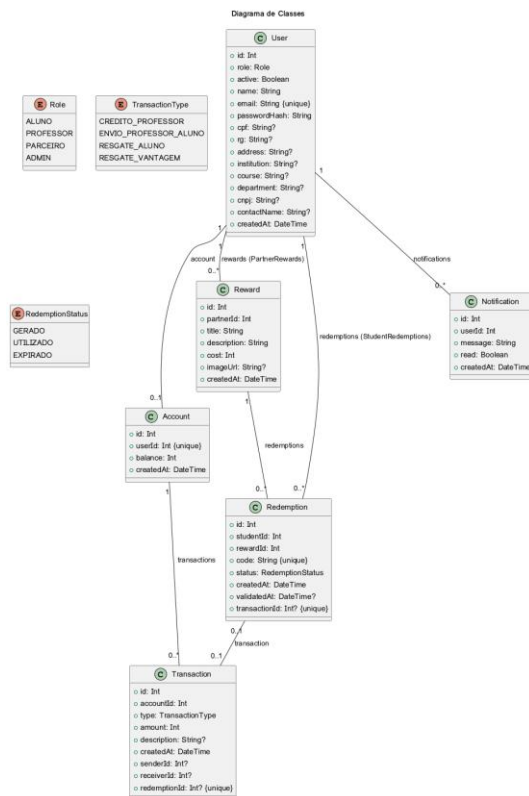


Figura 6: Diagrama de Classes

As classes principais são:

- **User:** Representa os atores (Aluno, Professor, Parceiro, Admin) com seus dados e papel (Role).
- **Account:** Armazena o saldo (balance) e o histórico de transações do usuário.
- **Reward:** Representa as vantagens cadastradas pelos Parceiros.
- **Redemption:** Representa o resgate de uma vantagem por um Aluno, incluindo o código do cupom e o status (RedemptionStatus).
- **Transaction:** Registra todas as movimentações de moedas (TransactionType).
- **Notification:** Armazena as notificações para os usuários.

3.4 Diagramas de Sequência

Os Diagramas de Sequência (Figuras 2, 3 e 4) já foram apresentados na Seção 2.3 e detalham a ordem temporal das interações entre os objetos do sistema para a realização dos Casos de Uso: Envio de Moedas, Resgate de Vantagem e Validação de Cupom.

3.5 Diagramas de Comunicação

Os Diagramas de Comunicação, embora não fornecidos explicitamente no repositório, representariam as mesmas interações dos Diagramas de Sequência, mas focando nos objetos e seus links de comunicação, em vez da ordem temporal.

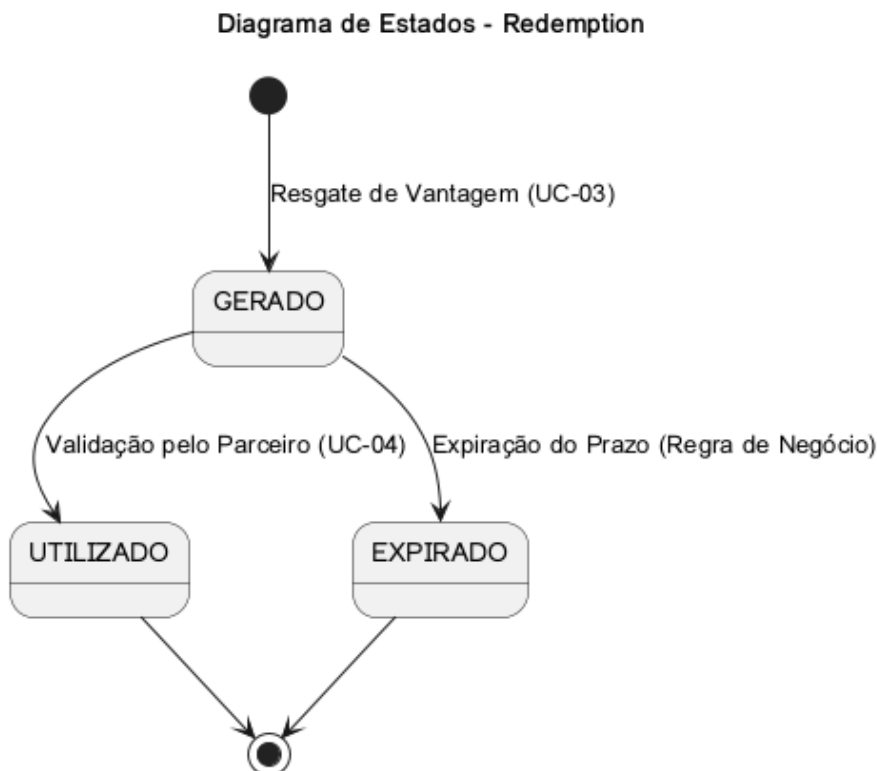
3.6 Diagramas de Estados

O sistema possui um estado explícito para o objeto Redemption (Resgate) e para o objeto User (ativo/inativo).

Diagrama de Estados para Resgate (Redemption)

O estado de um resgate evolui da seguinte forma:

Figura 7: Diagrama de Estados - Redemption



Estado	Descrição	Transição
GERADO	O cupom foi criado após o resgate pelo Aluno.	Validação pelo Parceiro
UTILIZADO	O cupom foi validado pelo Parceiro e a vantagem foi entregue.	-
EXPIRADO	O cupom perdeu a validade (regra de negócio não detalhada, mas presente no modelo).	-

Diagrama de Estados para Usuário (User)

O estado de um usuário é binário:

Estado	Descrição	Transição
true	Usuário ativo no sistema.	Desativação (Admin)
false	Usuário inativo no sistema.	Ativação (Admin)

4. Modelos de Dados

O modelo de dados é baseado no esquema do **Prisma ORM**, que utiliza um banco de dados **PostgreSQL**. O esquema define as entidades e seus relacionamentos, conforme detalhado abaixo.

Esquema do Banco de Dados (Prisma Schema)

```
``prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum Role {
  ALUNO
  PROFESSOR
  PARCEIRO
  ADMIN
}

enum TransactionType {
  CREDITO_PROFESSOR
  ENVIO_PROFESSOR_ALUNO
  RESGATE_ALUNO
  RESGATE_VANTAGEM
}

enum RedemptionStatus {
  GERADO
  UTILIZADO
  EXPIRADO
}
```

```

model User {
  id      Int    @id @default(autoincrement())
  role    Role
  active  Boolean @default(true)
  name    String
  email   String @unique
  passwordHash String
  cpf     String?
  rg      String?
  address String?
  institution String?
  course  String?
  department String?
  cnpj    String?
  contactName String?
  createdAt DateTime @default(now())

  // relações
  account      Account?
  rewards      Reward[] @relation("PartnerRewards")
  redemptions  Redemption[] @relation("StudentRedemptions")
  notifications Notification[] // campo adicionado
}

```

```

model Account {
  id      Int    @id @default(autoincrement())
  user    User    @relation(fields: [userId], references: [id])
  userId  Int     @unique
  balance Int     @default(0)
  createdAt DateTime @default(now())
}

```

```

transactions Transaction[]
}

```

```

model Reward {
  id      Int    @id @default(autoincrement())
  partner User    @relation("PartnerRewards", fields: [partnerId], references: [id])
  partnerId Int
  title   String
  description String
  cost    Int
  imageUrl String?
  createdAt DateTime @default(now())
}

```

```

redemptions Redemption[]
}

```

```

model Redemption {
  id      Int      @id @default(autoincrement())
  student User      @relation("StudentRedemptions", fields: [studentId], references:
[id])
  studentId Int
  reward   Reward   @relation(fields: [rewardId], references: [id])
  rewardId Int
  code     String   @unique
  status   RedemptionStatus @default(GERADO)
  createdAt DateTime @default(now())
  validatedAt DateTime?

transaction Transaction?
}

model Transaction {
  id      Int      @id @default(autoincrement())
  account Account   @relation(fields: [accountId], references: [id])
  accountId Int
  type    TransactionType
  amount  Int
  description String?
  createdAt DateTime @default(now())

// Relações opcionais para rastreamento
  senderId Int?
  receiverId Int?
  redemption Redemption? @relation(fields: [redemptionId], references: [id])
  redemptionId Int? @unique
}

model Notification {
  id      Int      @id @default(autoincrement())
  user    User      @relation(fields: [userId], references: [id])
  userId  Int
  message String
  read    Boolean   @default(false)
  createdAt DateTime @default(now())
}

```

Estratégias de Mapeamento Objeto-Relacional

O projeto utiliza o **Prisma ORM** para o mapeamento Objeto-Relacional (O/R). O Prisma atua como uma camada de abstração, convertendo os modelos definidos no schema.prisma (como User, Account, Reward, etc.) em tabelas no banco de dados PostgreSQL e vice-versa.

- **Entidades e Tabelas:** Cada model no Prisma (ex: model User) é mapeado diretamente para uma tabela no banco de dados.

- **Atributos e Colunas:** Os campos dos modelos (ex: name String) são mapeados para colunas na tabela, com o tipo de dado correspondente.
- **Chaves Primárias e Estrangeiras:** O campo @id define a chave primária. Relacionamentos (ex: user User @relation(...)) definem chaves estrangeiras, garantindo a integridade referencial.
- **Relacionamentos:** O Prisma gerencia relacionamentos de um-para-um (Account?), um-para-muitos (Reward[]) e muitos-para-muitos (implícitos ou explícitos) de forma declarativa. Por exemplo, o relacionamento entre User e Account é de um-para-um, onde Account tem uma chave estrangeira única (userId) referenciando User.

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Matheus Santos	16/11/2025	Criação inicial do documento com base no template e análise do repositório GitHub.	1.0

Referências

- [1] Repositório GitHub do Sistema de Moeda Estudantil. *viniciusmazzoli/Sistema-de-Moeda-Estudantil*. Disponível em: <https://github.com/viniciusmazzoli/Sistema-de-Moeda-Estudantil>.
- [2] Esquema do Banco de Dados do Sistema de Moeda Estudantil. *schema.prisma*. Disponível em: `/home/ubuntu/prisma_schema.txt`.